

report - project 2 - Threads, Parallelism and Concurrency

team:

Mika Yamin 325115996

Bar Hana Yehezkel 322723206

Q1. Amdahl's Law

1.

Parallel portion $(1-S) = 0.75$

Sequential portion $(S) = 1-0.75=0.25$

$$N = 4, \text{Speedup} = 0.25 + 4 \cdot 0.75 = 0.25 + 3 = 3.25 \approx 2.28$$

2.

Parallel portion $(1-S) = 0.60$

Sequential portion $(S) = 0.40$

Target Speedup = 3 According to Amdahl's Law, the maximum possible speedup even with an infinite number of processors is $\frac{1}{S}$. $\text{Max Speedup} = \frac{1}{0.40} = 2.5$ Because the absolute maximum theoretical speedup is 2.5, it is impossible to achieve a speedup of 3, regardless of how many processors are used.

3.

Parallel portion $(1-S) = 0.90$

Sequential portion $(S) = 0.10$

$$N = 32, \text{Speedup} = \frac{1}{0.10 + \frac{0.9}{32}} = \frac{1}{0.128125} \approx 7.80$$

4.

Sequential portion $(S) = 1-0.85=0.15$

$N \rightarrow \infty$ As the number of processors approaches infinity, the term $\frac{1-S}{N}$ approaches 0, so the speedup approaches $\frac{1}{S}$. $\text{Max Speedup} = \frac{1}{0.15} \approx 6.67$.

5.

Parallel portion $(1-S) = 0.80$

Sequential portion $(S) = 0.20$

$$\text{Calculation for 4 processors (N=4): } \text{Speedup}_4 = \frac{1}{0.20 + \frac{0.80}{4}} = \frac{1}{0.40} = 2.5$$

$$\text{Calculation for 32 processors (N=32): } \text{Speedup}_{32} = \frac{1}{0.20 + \frac{0.80}{32}} = \frac{1}{0.225} \approx 4.44$$

Conclusion: When the number of processors is increased from 4 to 32, the speedup increases from 2.5 to approximately 4.44. This means the performance improves, but it does not scale linearly (a 8× increase in processors only results in about a 1.77× actual speed improvement) due to the sequential bottleneck.

Q2. Process vs. Thread

1.Results:

Process Time (ms)	Thread Time (ms)
156.7110	12.4817
118.7236	0.4174
104.7735	0.3973
126.2868	6.9700
178.0342	12.8436
165.0530	13.3066
144.3959	13.3066
144.3959	2.2282
188.8097	17.5647
125.1168	0.6875

```
--- Launching Processes ---
[From Process] Hello World 9108
[From Process] Hello World 2047
[From Process] Hello World 6569
[From Process] Hello World 2587
[From Process] Hello World 2825
[From Process] Hello World 7177
[From Process] Hello World 3754
[From Process] Hello World 6222
[From Process] Hello World 6568
[From Process] Hello World 4834

--- Launching Threads ---
[From Thread] Hello World 3557
[From Thread] Hello World 5486
[From Thread] Hello World 8225
[From Thread] Hello World 8032
[From Thread] Hello World 5556
[From Thread] Hello World 6828
[From Thread] Hello World 5478
[From Thread] Hello World 6249
[From Thread] Hello World 3190
```

2. Explanation:

The results clearly show that creating a thread is way faster than creating a process (almost 100 times faster). This happens because a Process is heavyweight so the operating system has to allocate a completely new and isolated memory space (address space), page tables, and resources for it from scratch. On the other hand, a Thread is super lightweight because it runs inside an existing process. It doesn't need its own isolated memory, it just shares the parent process's heap, memory, and resources, and only gets a small private stack for itself.

Q3. Multithreaded Matrix Multiplication

Task Manager:

LNBITSSvc.exe	5064	Running	SYSTEM	00	-124 K	12	64 bit	Not allowed
Lsalso.exe	1124	Running	SYSTEM	00	0 K	1	64 bit	Not allowed
Isass.exe	1136	Running	SYSTEM	00	0 K	16	64 bit	Not allowed
M365Copilot.exe	19160	Running	Mika	00	0 K	35	64 bit	Disabled
MatrixMultiplication....	22100	Running	Mika	00	0 K	28	64 bit	Disabled

Name	PID	Status	User name	CPU	Worki...	Threads	Platform	UAC vii
MatrixMultiplication....	6048	Running	Mika	00	0 K	28	64 bit	Disabl

Code in the zip file

Q4 .Multithreaded MergeSort

a. Our multi-threaded merge-sort algorithm is designed based on the "Fork and Join" parallelism strategy. It recursively divides the sorting workload into smaller sub-tasks, processes them concurrently using multiple threads, and then synchronizes the threads to merge the results.

The algorithm operates in the following steps:

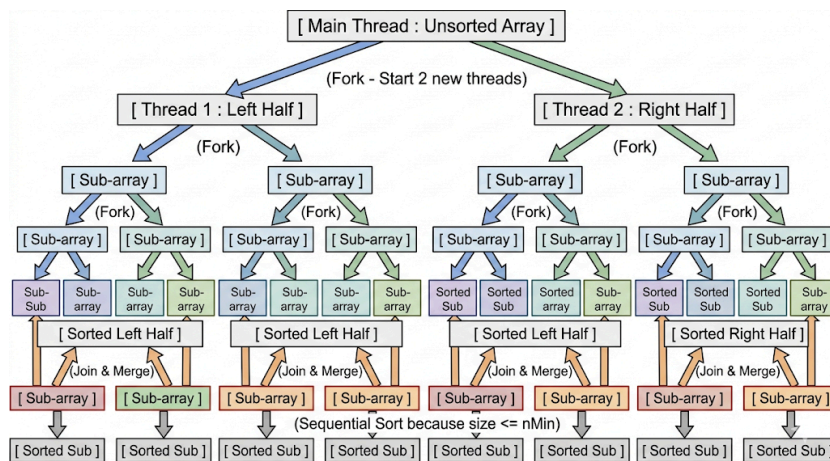
1. Threshold Check (Base Case): Creating a new thread involves Operating System overhead, such as allocating a Thread Control Block (TCB) and managing context switches. If we were to open a new thread for every single element, this overhead would take longer than the sorting process itself. Therefore, the algorithm uses a threshold parameter (n_{Min}). If the size of the given array is less than or equal to n_{Min} , the algorithm stops opening new threads and simply sorts the array using a standard, sequential sorting method.

2. Fork (Splitting Phase): If the array is larger than n_{Min} , the main (parent) thread divides the array into two equal halves: a left sub-array and a right sub-array. The parent thread then spawns two new child threads. Each child thread is assigned the task of recursively executing the MergeSort function on its respective half. Once the Start() method is called, both threads run concurrently on the CPU.

3. Join (Synchronization Phase): The parent thread cannot merge the arrays before they are fully sorted. Therefore, it calls the Join() method on both child threads. This is a blocking operation that halts the parent thread, forcing it to wait until both the left and right threads have completely finished their execution and terminated.

4. Merge Phase: Once both child threads have successfully joined, the parent thread knows that both halves are now fully sorted. It then merges the two sorted lists into a single, sorted list. The comparison of the elements during the merge is done using the built-in string.Compare() function, ensuring alphabetical order. Finally, the merged list is returned to the caller.

b.



c+d1. Code in the zip file

d2. Here is the output showing the multithreaded merge sort efficiently sorting:

1,000,000 elements in 515ms:

Generating a large array of random strings...

Starting Multithreaded MergeSort on 1000000 elements...

Sorting completed!

Total time taken: 515 ms

First 5 elements of sorted list:

10000
100000
100001
100002
100002

Process finished with exit code 0.

Q5. Parallel Image Search

Code in the zip file.